CS336 作业 5（对齐）：对齐与推理 RL 版本 1.0.2

CS336 教师团队

2025 年春季

1 作业概述

在这个作业中，你将获得一些实际操作经验，学习如何训练语言模型在对齐时解决数学问题。

你将实现的内容。

1. 零样本提示基线用于竞赛数学问题MATH数据集 Hendrycks等人。[2021].
2. 有监督微调，使用来自更强推理模型（DeepSeek R1，DeepSeek-AI等2025年）的推理轨迹。
3. 专家迭代用于通过验证奖励提升推理性能。
4. 基于组相对策略优化（GRPO）用于通过验证奖励提升推理性能。科室。

对感兴趣的人，我们将有一项完全可选的作业部分，用于将语言模型与人类偏好对齐，该部分将在未来几天发布。

您将运行的内容。

1. 测量 Qwen 2.5 Math 1.5B 零样本提示性能（我们的基线）。
2. 使用来自 R1 的推理轨迹在 Qwen 2.5 Math 1.5B 上运行 SFT。
3. 使用经过验证的奖励在 Qwen 2.5 Math 1.5B 上运行专家迭代。
4. 在 Qwen 2.5 Math 1.5B 上运行 GRPO，并使用已验证的奖励。

代码长什么样。所有作业代码以及这份文档都可以在 GitHub 上找到：

`github.com/stanford-cs336/assignment5-alignment`

请 `git clone` 这个仓库。如果有任何更新，我们会通知你，你可以 `git pull` 来获取最新版本。

1. `cs336_alignment/*`：这是你编写作业 5 代码的地方。请注意这里没有代码（除了少量启动代码），所以你应该能够从头开始做任何你想做的事情。

2. `cs336_alignment/prompts/*`: 为了方便您使用，我们提供了带有提示的文本文件，以尽量减少从 PDF 复制粘贴提示到您的代码时可能出现的错误。

3. `tests/*.py`: 这包含所有你必须通过的测试。你只需要通过 **`tests/test_sft.py`** 和 **`tests/test_grpo.py`** 中的测试——其余的测试是用于作业的非强制性部分。这些测试调用了在 `tests/adapters.py` 中定义的钩子。你将实现适配器以将你的代码连接到这些测试。编写更多测试和/或修改测试代码有助于调试你的代码，但你的实现被期望通过最初提供的测试套件。

4. `README.md`：这个文件包含一些关于设置你环境的基本说明。

你能使用什么。我们期望你从头开始构建大部分的RL相关组件。你可以使用vLLM等工具从语言模型生成文本（§3.1）。此外，你可以使用HuggingFace Transformers加载Qwen 2.5 Math 1.5B模型和分词器并运行前向传递（§4.1），但你不能使用任何训练工具（例如， `Trainer` 类）。

如何提交。您将向Gradescope提交以下文件：

- `writeup.pdf`：回答所有书面问题。请排版您的答案。
- `code.zip`：包含您编写的所有代码。

2 语言模型推理

2.1 动机

语言模型的一个显著用例是构建能够处理广泛自然语言处理任务的通用系统。在本作业中，我们将关注语言模型的一个开发用例：数学推理。它将作为我们设置评估、执行监督微调以及通过强化学习（RL）教授语言模型进行推理的试验平台。

我们过去做作业的方式将会有两个不同之处。

- 首先，我们将不再使用我们之前使用的语言模型代码库和模型。我们理想情况下希望使用从之前作业中训练的基础语言模型，但微调这些模型将不会给我们一个令人满意的结果——这些模型太弱了，无法展示非平凡的数学推理能力。由于这个原因，我们将切换到一个现代、高性能的语言模型（Qwen 2.5 Math 1.5B Base），并在该模型之上进行大部分工作。

- 其次，我们将引入一个新的基准来评估我们的语言模型。上
到目前为止，我们一直认为交叉熵是许多下游任务的良好替代品。然而，这个作业的目的是弥合基础模型和下游任务之间的差距，因此我们将不得不使用与交叉熵无关的评估方法。我们将使用 Hendrycks 等人开发的 MATH 12K 数据集 {v1}，其中包含具有挑战性的高中竞赛数学问题。我们将通过比较语言模型输出来评估它们。

  下游任务，因此我们将需要使用与交叉熵分离的评估方法。我们将使用 Hendrycks 等人 [2021]，提供的 MATH 12K 数据集，该数据集包含具有挑战性的高中竞赛数学问题。我们将通过比较语言模型输出来进行评估

  与参考答案对比。

2.2 思维链推理与推理式强化学习

语言模型领域近期一个令人兴奋的趋势是使用思维链推理来提升多种任务的表现。思维链指的是逐步思考问题的过程，在得出最终答案前会生成中间推理步骤。

使用大型语言模型的思维链推理。早期的思维链方法通过使用“草稿纸”将问题分解为中间步骤来微调语言模型，以解决简单的数学任务如算术运算 [Nye等人， 2021]。其他研究则提示强大的模型在回答前“逐步思考”，发现这显著提升了数学推理任务（如小学数学问题）的表现 [Wei等人， 2023]。

通过专家迭代进行推理。自教学推理器（STaR） [Zelikman 等人， 2022]将推理视为一个引导循环：预训练模型首先采样多样化的思维链（CoTs），仅保留那些能得出正确答案的，然后在这些“专家”轨迹上进行微调。迭代这个循环可以提高 LM 的推理能力和解决率。STaR 证明，这种专家迭代 [Anthony 等人， 2017] 通过自动的、基于字符串匹配的生成答案验证，可以在没有人类编写的推理轨迹的情况下引导推理技能。

使用验证奖励、o1 和 R1 的推理强化学习。近期研究探索了使用更强大的强化学习算法和验证奖励来提高推理性能。OpenAI 的 o1（及其后续的 o3/o4） [OpenAI 等人， 2024], DeepSeek 的 R1 [DeepSeek-AI 等人， 2025], 以及 Moonshot 的 kimi k1.5[团队等， 2025] 使用策略梯度方法 [Sutton 等人， 1999] 在数学和代码任务上进行训练，其中通过字符串匹配或单元测试验证正确性，在竞赛数学和编码性能上取得了显著改进。后续工作如 Open-R1 [Face， 2025], SimpleRL-Zoo [Zeng 等人， 2025],和 TinyZero [Pan 等人， 2025] 证实，即使模型参数量小至 1.5B，纯强化学习配合验证奖励也能提高推理性能。

我们的设置：模型和数据集。在以下部分，我们将考虑逐步更复杂的方法来训练一个基础语言模型，使其能够逐步推理以解决数学问题。对于这个作业，我们将使用Qwen 2.5 Math 1.5B基础模型，该模型在高质量合成数学预训练数据 [Yang等人， 2024]上持续预训练而成。MATH数据集可在Together集群上获取 `/data/a5-alignment/MATH`。

开源审计员提示：替代数据集

不幸的是，由于版权声明，MATH数据集无法公开获取。如果你在家中跟随学习，可以使用以下开源数学推理数据集之一：

- 倒计时 [Pan等人， 2025], 在此处可用：一个基于英国电视节目《倒计时》的简单合成任务，它已成为小规模推理RL的流行测试平台。
- GSM8K [Cobbe 等人，2021a], 可在以下链接获取：小学数学问题，这些题目比 MATH 更简单，但应能帮助你调试正确性并熟悉推理 RL 管道。
- Tulu 3 SFT 数学 [Lambert 等人， 2025], 可在以下链接获取：使用 GPT-4o 和 Claude 3.5 Sonnet 生成的合成数学问题。由于这些是合成的，部分答案（甚至问题本身）可能不完全正确。
- 其他一些数学 SFT 数据集链接在此处。

若未直接提供，要获取短地面真实标签（例如 `1/2`），你可以使用 Math-Verify 等数学答案解析器处理地面真实列。

3 测量零样本 MATH 性能

我们将从评估我们的基础语言模型在MATH的5K示例测试集上的性能开始。建立这个基线有助于理解后续每种方法如何影响模型行为。

除非另有说明，对于MATH上的实验，我们将使用DeepSeek R1-Zero模型的以下提示 [DeepSeek-AI等, 2025]。我们将将其称为 `r1_zero` 提示：

```
A conversation between User and Assistant. The User asks a question, and the Assistant
solves it. The Assistant first thinks about the reasoning process in the mind and ,→
then provides the User with the answer. The reasoning process is enclosed within ,→
<think> </think> and answer is enclosed within <answer> </answer> tags, respectively, ,→i.e.,
 <think> reasoning process here </think> <answer> answer here </answer>. ,→User: {question}

Assistant: <think>
```

The `r1_zero`提示位于文本文件 `cs336_alignment/prompts/r1_zero.prompt` 中。

在提示中， `question` 指的是我们插入的一些问题（例如， `Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?`）。期望是模型扮演助手的角色，并开始生成思考过程（因为我们已经包含了一个左思考标签 `<think>`），然后用 `</think>` 结束思考过程，并在答案标签内生成最终符号答案，如 `<answer> 4x + 10 </answer>`。让模型生成 `<answer> </answer>` 标签的目的是为了我们能够轻松解析模型的输出并与真实答案进行比较，并且在我们看到正确的答案标签 `</answer>` 时停止响应生成。

关于提示词选择的说明。结果表明，在 RL 后， `r1_zero` 提示词并非最大化下游性能的最佳选择，因为提示词与 Qwen 2.5 Math 1.5B 模型预训练方式之间存在不匹配。刘等人 [2025] 发现，仅用问题提示模型（且不添加其他内容）即可从非常高的准确率开始，例如，在 100+ 步 RL 后就能匹配 `r1_zero` 提示词。他们的研究结果表明，Qwen 2.5 Math 1.5B 模型在预训练时已经包含了这类问答对。

尽管如此，我们为本次作业选择 `r1_zero` 提示词，因为在少量步骤内使用 RL 就能显示出明显的准确率提升，这使我们能够快速了解 RL 的工作机制并验证正确性，即使最终性能并非最优。作为现实检验，你将在作业后续直接与 `question_only` 提示词进行比较。

3.1 使用 vLLM 进行离线语言模型推理

为了评估我们的语言模型，我们需要为各种提示生成续写（回复）。虽然当然可以自己实现生成函数（例如，你在作业1中那样做），但强化学习的高效实现需要高性能推理技术，而实现这些推理技术超出了本次作业的范围。因此，在本作业中，我们将推荐使用vLLM进行离线批量推理。vLLM是一个高吞吐量且内存高效的语言模型推理引擎，它集成了多种有用的效率技术（例如，优化的CUDA内核、用于高效注意力KV缓存的PagedAttention [Kwon等人， 2023], 等）。要使用vLLM为一系列提示生成续写:[1]

```python
from vllm import LLM, SamplingParams

# Sample prompts.
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]
```

[1]示例来自 https://github.com/vllm-project/vllm/blob/main/examples/offline_推理.py。

```python
# Create a sampling params object, stopping generation on newline.
sampling_params = SamplingParams(
    temperature=1.0, top_p=1.0, max_tokens=1024, stop=["\n"]
)

# Create an LLM.
llm = LLM(model=<path to model>)

# Generate texts from the prompts. The output is a list of RequestOutput objects
# that contain the prompt, generated text, and other information.
outputs = llm.generate(prompts, sampling_params)

# Print the outputs.
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

在上面的示例中， `LLM` 可以初始化为HuggingFace模型的名称（如果本地未找到，将自动下载并缓存），或HuggingFace模型的路径。由于下载可能需要很长时间（尤其是对于较大的模型，例如70B参数），并且为了节省集群磁盘空间（以免每个人都拥有预训练模型的独立副本），我们在Together集群上下载了以下预训练模型，并存储在以下路径。请不要在Together集群上重新下载这些模型：

- Qwen 2.5 数学 1.5B 基础版（用于推理实验）：
  `/data/a5-alignment/models/Qwen2.5-Math-1.5B`
- Llama 3.1 8B 基础版（用于可选指令微调实验）：
  `/data/a5-alignment/models/Llama-3.1-8B`
- Llama 3.3 70B 指令版（用于可选指令微调实验）：
  `/data/a5-alignment/models/Llama-3.3-70B-Instruct`

3.2 零样本 MATHBaseline

提示设置。为了评估模型在 MATH 测试集上的零样本性能，我们将直接加载示例并提示语言模型使用上述 `r1_zero` 提示来回答问题。

评估指标。当我们评估选择题或二元响应任务时，评估指标很明确——我们测试模型是否输出完全正确的答案。

在数学问题中，我们假设存在一个已知的真实值（例如0.5），但我们不能简单地测试模型是否输出正好0.5——它也可能回答 `<answer> 1/2 </answer>`。由于这一点，我们在评估MATH时必须处理一个棘手的问题：如何匹配来自大模型的语义等价响应。

为此，我们希望提出一种答案解析函数，该函数以模型的输出和已知的真实值作为输入，并返回一个布尔值以指示模型的输出是否正确。例如，一个奖励函数可以接收以 `<answer> She sold 15 clips. </answer>` 结尾的模型字符串输出和黄金答案 72，如果模型的输出正确则返回 `True` ，否则返回 `False` （在这种情况下，它应该返回 `False` ）。

在我们的 MATH 实验中，我们将使用最近在推理 RL [Liu 等人 2025]工作中使用的快速且相当准确的答案解析器。这个奖励函数在 `cs336_alignment.drgrpo_grader.r1_zero_reward_fn` 中实现，除非另有说明，否则您应该使用它来评估 MATH 的性能。

生成超参数。在生成回复时，我们将使用温度1.0、top-p 1.0、最大生成长度1024进行采样。提示要求模型以字符串 `</answer>`结束其答案，因此我们可以指示vLLM在模型输出此字符串时停止：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params.stop = ["</answer>"]
sampling_params.include_stop_str_in_output = True
```

问题 (math_baseline)：4分

(a) 编写一个脚本评估Qwen 2.5 Math 1.5B零样本性能在MATH上。此脚本应 (1) 从 `/data/a5-alignment/MATH/validation.jsonl`加载MATH验证示例，
(2) 使用 `r1_zero` 提示将它们格式化为语言模型的字符串提示，以及 (3) 为每个示例生成输出。此脚本还应 (4) 计算评估指标并
(5) 将示例、模型生成结果和相应的评估分数序列化到磁盘上，以便在后续问题中进行分析。

对于您的实现，包含一个类似以下参数的方法 `evaluate_vllm` 可能会有所帮助，因为您以后可以重用它：

```
def evaluate_vllm(
    vllm_model: LLM,
    reward_fn: Callable[[str, str], dict[str, float]],
    prompts: List[str],
    eval_sampling_params: SamplingParams
) -> None:
    """
    Evaluate a language model on a list of prompts,
    compute evaluation metrics, and serialize results to disk.
    """
```

可交付成果：一个用于评估基准零样本MATH性能的脚本。

(b) 在Qwen 2.5 Math 1.5B上运行你的评估脚本。有多少模型生成结果属于以下类别：(1) 格式和答案奖励均为1的正确结果，(2) 格式奖励为1且答案奖励为0的结果，(3) 格式奖励为0且答案奖励为0的结果？观察至少10个格式奖励为0的案例，你认为问题出在基础模型的输出上，还是解析器上？为什么？在至少10个格式奖励为1但答案奖励为0的案例中，情况又如何？

可交付成果：关于模型和奖励函数性能的评论，包括示例。
每个类别的。

(c) Qwen 2.5 Math 1.5B零样本基准在MATH上的表现如何？
可交付成果：1-2句包含评估指标的内容。

4 MATH的监督微调

Algorithm 1 Supervised Finetuning (SFT)

Input initial policy model $\pi_{\theta_{\text{init}}}$; SFT dataset $\mathcal{D}$

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for step = 1, ..., `n_sft_steps` **do**
    Sample a batch of question-response pairs $\mathcal{D}_b$ from $\mathcal{D}$
    Compute the cross-entropy loss of the responses given the questions using the model π_θ
    Update the model parameters θ by taking a gradient step with respect to the cross-entropy loss
end for

Output π_θ

用于推理的监督微调。在本节中，我们将基于MATH数据集（算法1）微调我们的基础模型。由于我们的目标是提高模型的推理能力，而不是直接微调它以预测正确答案，我们将微调它以首先生成一个思维链推理轨迹，然后生成答案。为此，我们已提供这样一个推理轨迹的数据集，该数据集来自DeepSeek R1 DeepSeek-AI等。 [2025], 在

`/data/a5-alignment/MATH/sft.jsonl`

在实际训练推理模型时，SFT常被用作第二次RL微调步骤的预热。这主要有两个原因。首先，SFT需要高质量的标注数据（即带有预存推理轨迹的数据），而RL只需要正确答案作为反馈。其次，即使在标注数据丰富的场景下，RL也能通过找到比SFT数据更好的策略来提升性能。不幸的是，我们使用的模型规模不够大，无法在组合SFT和RL时展示效果，因此在这个作业中我们将将这两个阶段分开处理。

4.1 使用HuggingFace模型

加载HuggingFace模型和分词器。要从本地目录（在 `bfloat16` 中且使用FlashAttention-2节省内存）加载HuggingFace模型和分词器，你可以使用以下启动代码：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "/data/a5-alignment/models/Qwen2.5-Math-1.5B",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained("/data/a5-alignment/models/Qwen2.5-Math-1.5B")
```

前向传播。在加载模型后，我们可以在一批输入ID上运行前向传播，并获取输出中的logits（带有 `.logits` 属性）。然后，我们可以计算模型预测的logits与实际标签之间的损失：

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)

logits = model(input_ids).logits
loss = F.cross_entropy(..., ...)
```

保存训练好的模型。训练完成后，要将模型保存到指定目录，可以使用.save_pretrained() 函数，传入目标输出目录的路径。确保保存到/data/yourusernamе 下，因为它们可能相当大。我们还建议将分词器也保存下来（即使你没有修改它），这样模型和分词器就是自包含的，可以从单个目录加载。

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

梯度累积。尽管在 bfloat16 中加载模型并使用 FlashAttention-2，即使是 80GB 的 GPU 也不足以支持合理的批处理大小。为了使用更大的批处理大小，我们可以使用一种称为梯度累积的技术。梯度累积的基本思想是，而不是在每个批次后更新我们的模型权重（即执行优化器步骤），我们会在多个批次上累积梯度，然后再执行梯度步骤。直观地说，如果我们有更大的 GPU，那么一次性计算 32 个示例的梯度的结果应该与将它们分成 16 个包含 2 个示例的批次，然后最后取平均值的结果相同。

在 PyTorch 中实现梯度累积非常简单。回想一下，每个权重张量都有一个 .grad 属性来存储其梯度。在我们调用 loss.backward() 之前， .grad 属性为 None。在我们调用 loss.backward() 之后， .grad 属性包含梯度。通常，我们会执行优化器步骤，然后使用 optimizer.zero_grad() 将梯度归零，这会重置权重张量的 .grad 字段：

```
for inputs, labels in data_loader:
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels)

    # Backward pass.
    loss.backward()

    # Update weights.
    optimizer.step()
    # Zero gradients in preparation for next iteration.
    optimizer.zero_grad()
```

要实现梯度累积，我们只需每隔 k 步调用 optimizer.step() 和 optimizer.zero_grad()，其中 k 是梯度累积的步数。在调用 loss.backward() 之前，我们将损失除以 gradient_accumulation_steps，以便梯度在梯度累积的步数之间进行平均。

```
gradient_accumulation_steps = 4
for idx, (inputs, labels) in enumerate(data_loader):
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels) / gradient_accumulation_steps

    # Backward pass.
    loss.backward()

    if (idx + 1) % gradient_accumulation_steps == 0:
        # Update weights every `gradient_accumulation_steps` batches.
        optimizer.step()
```

```
# Zero gradients every `gradient_accumulation_steps` batches.
optimizer.zero_grad()
```

因此，在训练时，我们的有效批大小乘以 k，即梯度累积步数。

4.2 SFT辅助方法

接下来，我们将实现一些您在 SFT 和后续 RL 实验中会使用的辅助方法。关于命名的一个简短说明：在以下章节中，我们将交替使用“输出”、“完成”或“响应”来指代模型根据提示生成的完成。

对提示和输出进行分词。对于每一对问题和目标输出（q、o），我们将分别对问题和输出进行分词，然后将它们连接起来。然后，我们可以使用我们的SFT模型（或在后续章节中，我们的RL策略）对输出的对数概率进行评分。此外，我们需要构建一个`response_mask`：一个布尔掩码，对于响应中的所有标记是 `True`，对于所有问题和填充标记是 `False`。我们将在这个训练循环中使用这个掩码，以确保我们只对响应标记计算损失。

问题 (`tokenize_prompt_and_output`)：提示和输出分词 **(2 分)**

交付物：实现一个方法 `tokenize_prompt_and_output`，将问题和输出分别分词，然后连接在一起，并构建一个 `response_mask`。推荐使用以下接口：

`def tokenize_prompt_and_output(prompt_strs, output_strs, tokenizer):` 对提示和输出字符串进行分词，并构建一个掩码，响应词标记为 1，其他词（提示或填充词）标记为 0。

Args:

`prompt_strs: list[str]` List of prompt strings.

`output_strs: list[str]` List of output strings.

`tokenizer: PreTrainedTokenizer` Tokenizer to use for tokenization.

返回：

`dict[str, torch.Tensor]`. 设 `prompt_and_output_lens` 是一个包含分词后提示和输出字符串长度的列表。那么返回的字典应该包含以下键：

`input_ids` torch.Tensor of shape `(batch_size, max(prompt_and_output_lens) - 1)`: 分词后的提示和输出字符串，最后一个token被切掉了。

`labels` torch.Tensor形状为 `(batch_size, max(prompt_and_output_lens) - 1)`: 偏移的输入id，即不包含第一个token的输入id。

`response_mask` torch.Tensor形状为 `(batch_size, max(prompt_and_output_lens) - 1)`: 一个应用于标签中响应token的掩码。

要测试你的代码，请实现 `[adapters.run_tokenize_prompt_and_output]`。然后，使用 `uv run pytest -k test_tokenize_prompt_and_output` 运行测试，并确保你的实现通过它。

记录每个token的熵。在进行强化学习（RL）时，跟踪每个token的熵通常很有用，以查看模型的预测分布是否变得（过于）自信。我们现在将实现这一点，并比较我们的微调方法如何影响模型的预测熵。

离散分布 $p(x)$ 在支撑集 $\mathcal{X}$ 上的熵定义为

$$H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x). \tag{1}$$

根据我们的 SFT 或 RL 模型的 logits，我们将计算每个 token 的熵，即每个下一个 token 预测的熵。

问题 (`compute_entropy`): 每个token的熵 **(1 point)**

交付物：实现一个方法 `compute_entropy`，用于计算下一个 token 的预测。
推荐以下接口：

```
def compute_entropy(logits: torch.Tensor) -> torch.Tensor:
```

获取下一个词预测的熵（即词汇维度上的熵）。

参数：

`logits: torch.Tensor` Tensor of shape `(batch_size, sequence_length, vocab_size)` containing unnormalized logits.

返回值：

`torch.Tensor` 形状 `(batch_size, sequence_length)`。每个下一个词预测的熵。预测。

注意：你应该使用数值稳定的计算方法（例如，使用 `logsumexp`）来避免溢出。要测试你的代码，请实现 `[adapters.run_compute_entropy]`。然后运行 `uv run pytest -k test_compute_entropy` 并确保你的实现通过。

从模型中获取对数概率。从模型中获取对数概率是一个原始操作，我们既需要在SFT（监督微调）中也需要在RL（强化学习）中使用它。

对于前缀 x、生成下一个词 logits 的语言模型 $f_\theta(x) \in \mathbb{R}^{|\mathcal{V}|}$ 以及标签 $y \in \mathcal{V}$，y 的对数概率是

$$\log p_\theta(y \mid x) = \log \left[\mathrm{softmax}(f_\theta(x))\right]_y, \tag{2}$$

其中符号 $[x]_y$ 表示向量 x 的第 y 个元素。

你需要使用一种数值稳定的计算方法来计算这个值，并且可以自由使用来自 `torch.nn.functional` 的方法。我们还建议包含一个可选参数来计算并返回词元的熵。

问题 (`get_response_log_probs`)：响应对数概率（和熵） **(2 points)**

交付物：实现一个方法 `get_response_log_probs`，它从因果语言模型中获取每个词元的条件对数概率（给定前面的词元），并且可选地获取模型下一个词元分布的熵。

以下界面建议使用：

```python
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数:

`model: PreTrainedModel` 用于评分的HuggingFace模型（放置在正确的设备上，并在不需要计算梯度时置于推理模式）。

`input_ids: torch.Tensor` 形状 (`batch_size, sequence_length`)，将提示符 +响应标记作为您的分词方法生成的标记。

`labels: torch.Tensor` 形状 (`batch_size, sequence_length`)，由您的分词方法生成的标签。

`return_token_entropy: bool` 如果 `True`，则通过调用`compute_entropy`返回每个标记的熵。

返回:

`dict[str, torch.Tensor]`.

`"log_probs"` 形状 (`batch_size, sequence_length`)，条件对数概率 $\log p_\theta(x_t \mid x_{<t})$.

`"token_entropy"` 可选，形状 (`batch_size, sequence_length`)，每个token的熵 for each position (present only if `return_token_entropy=True`).

实现技巧:

- Obtain logits with `model(input_ids).logits`.

要测试你的代码，实现 `[adapters.run_get_response_log_probs]`。然后运行 `uv run pytest -k test_get_response_log_probs` and ensure the test passes.

SFT微批处理训练步骤。在SFT中我们最小化的损失是给定提示的目标输出的负对数似然。为了计算这个损失，我们需要计算给定提示的目标输出的对数概率，并对输出中的所有token求和，同时掩码提示中的token和填充token。

我们将实现一个辅助函数来处理这个，稍后我们也会在RL过程中使用它。

问题 (`masked_normalize`): 掩码归一化 **(1 point)**

可交付成果：实现一个方法 `masked_normalize`，对张量元素求和并在归一化时使用常量，同时尊重布尔掩码。
以下界面建议使用:

```python
def masked_normalize(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    normalize_constant: float,
    dim: int | None = None,
) -> torch.Tensor:
```

对某个维度求和，并通过一个常数进行归一化，仅考虑满足 mask== 1 的元素。

参数：

tensor: torch.Tensor 待求和和归一化的张量。

mask: torch.Tensor 与 tensor 形状相同；位置为 1 的元素会被包含在求和中。

normalize_constant: float 用于归一化的常数。

dim: int | None 在归一化前沿哪个维度求和。如果为 None，则对所有维度求和。

返回：

torch.Tensor 归一化求和，其中被掩码的元素 (mask == 0) 不会对求和结果产生影响
求和结果产生
影响。

要测试您的代码，请实现 [adapters.run_masked_normalize]。然后运行 uv run pytest -k test_masked_normalize 并确保它通过。

SFT 微批次训练步骤。我们现在可以实现对 SFT 的单个微批次训练步骤（回想一下，对于训练小批量，如果 gradient_accumulation_steps > 为 1，我们会迭代多个微批次）。

问题 (sft_microbatch_train_step)：微批次训练步骤 (3 分)

可交付成果：为SFT实现单个微批处理更新，包括交叉熵损失、求和
以及掩码，并执行梯度缩放。
推荐以下接口：

```
def sft_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    normalize_constant: float = 1.0,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

在微批处理上执行前向和后向传递。

参数：

policy_log_probs (batch_size, sequence_length)，来自正在训练的SFT策略的每个token的log概率。

response_mask (batch_size, sequence_length)，响应token为1，提示/padding为0。

gradient_accumulation_steps 每个优化器步骤的微批次数。

normalize_constant 用于除以总和的常数。可以将其留为 1.0。

返回：

tuple[torch.Tensor, dict[str, torch.Tensor]].

损失标量张量。微批处理的损失值，已根据梯度累积进行调整。我们返回此值以便记录。

包含底层损失调用元数据的字典，以及任何您可能想要记录的其他统计信息的信息。

实现技巧：

- 您应该在这个函数中调用 `loss.backward()` 。确保调整梯度累积。累积。

要测试您的代码，请实现[adapters.run_sft_microbatch_train_step]。然后运行 `uv run pytest -k test_sft_microbatch_train_step` 并确认其通过。

循环记录生成过程。始终建议进行一些涉及模型生成的循环记录，而推理SFT/RL也不例外。编写一个函数 `log_generations`，提示您的模型为一些给定提示（例如，从验证集采样）生成响应。对于每个示例，记录以下内容是个好主意：

1. 输入提示。
2. SFT/RL模型生成的响应。
3. 真实答案。
4. 奖励信息，包括格式、答案和总奖励。
5. 响应的平均token熵。
6. 平均响应长度、正确响应的平均响应长度以及错误响应的平均响应长度。

问题 (`log_generations`): 记录生成内容 (1分)

交付物：实现一个函数 `log_generations` ，用于记录模型生成的内容。

4.3 SFT实验

使用上述片段，您现在将实现完整的SFT流程（算法1），在MATH数据集上微调Qwen 2.5 Math 1.5B基础模型。 `/data/a5-alignment/MATH/sft.jsonl`中的每个示例都由一个格式化的提示和目标响应组成，其中目标响应包括思维链推理轨迹和最终答案。特别是，每个示例都是类型为 `{"prompt": str, "response": str}`的JSON元素。

为了跟踪模型在训练过程中的进度，您应该定期在MATH验证集上评估它。您应该使用2个GPU运行脚本，一个GPU用于策略模型，另一个GPU用于vLLM实例来评估策略。为了使其工作，这里有一些启动代码，用于初始化vLLM，并在每个滚动阶段之前将策略权重加载到vLLM实例中：

```
from vllm.model_executor import set_random_seed as vllm_set_random_seed

def init_vllm(model_id: str, device: str, seed: int, gpu_memory_utilization: float = 0.85):
    """
    Start the inference process, here we use vLLM to hold a model on
    a GPU separate from the policy.
```

```
    """
    vllm_set_random_seed(seed)

    # Monkeypatch from TRL:
    # https://github.com/huggingface/trl/blob/
    #   22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py
    # Patch vLLM to make sure we can
    # (1) place the vLLM model on the desired device (world_size_patch) and
    # (2) avoid a test that is not designed for our setting (profiling_patch).
    world_size_patch = patch("torch.distributed.get_world_size", return_value=1)
    profiling_patch = patch(
        "vllm.worker.worker.Worker._assert_memory_footprint_increased_during_profiling",
        return_value=None
    )
    with world_size_patch, profiling_patch:
        return LLM(
            model=model_id,
            device=device,
            dtype=torch.bfloat16,
            enable_prefix_caching=True,
            gpu_memory_utilization=gpu_memory_utilization,
        )

def load_policy_into_vllm_instance(policy: PreTrainedModel, llm: LLM):
    """
    Copied from https://github.com/huggingface/trl/blob/
        22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py#L670.
    """
    state_dict = policy.state_dict()
    llm_model = llm.llm_engine.model_executor.driver_worker.model_runner.model
    llm_model.load_weights(state_dict.items())
```

You may find it helpful to log metrics with respect to both the train and validation steps (this will also be useful in later RL experiments). To do this in `wandb`, you can use the following code:

```
# Setup wandb metrics
wandb.define_metric("train_step")  # the x-axis for training
wandb.define_metric("eval_step")  # the x-axis for evaluation

# everything that starts with train/ is tied to train_step
wandb.define_metric("train/*", step_metric="train_step")

# everything that starts with eval/ is tied to eval_step
wandb.define_metric("eval/*", step_metric="eval_step")
```

Lastly, we suggest that you use gradient clipping with clip value 1.0.

问题 (`sft_experiment`)：在 MATH 数据集上运行 SFT **(2 points) (2 H100 hrs)**

1. 使用 Qwen 2.5 Math 1.5B 基础模型，在 `/data/a5-alignment/MATH/sft.jsonl` 中提供的推理 SFT 示例上运行 SFT，并改变 SFT 的唯一示例数量

范围 {128, 256, 512, 1024}，以及使用完整数据集。调整学习率和批大小，在使用完整数据集时，至少达到 15% 的验证准确率。
交付物：与不同数据集大小相关的验证准确率曲线。

2. 筛选推理 SFT 示例，仅保留产生正确答案的示例。在（完整）筛选数据集上运行 SFT，并报告筛选数据集的大小以及您实现的验证准确率。

交付物：报告数据集的大小以及您实现的验证准确率曲线。将您的发现与之前的 SFT 实验进行比较。

5 专家迭代用于 MATH

在上一节中，我们观察到可以通过从 SFT 数据中筛选出不良示例来提高我们的 SFT 模型的性能。在本节中，我们将更进一步：我们将应用此筛选过程到由我们基础模型本身生成的推理轨迹。该过程在文献中称为专家迭代 [Anthony 等人， 2017]，，在语言模型的背景下，已在 Cobbe 等人 [2021b],Zelikman 等人 [2022], Dohan 等人 [2022], Gulcehre 等人 [2023] 中进行探索。

Algorithm 2 Expert iteration (EI)

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward function R; task questions $\mathcal{D}$

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for step = 1, ..., `n_ei_steps` **do**
  Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$
  Set the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
  Sample G outputs $\{o^{(i)}\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
  Compute rewards $\{r^{(i)}\}_{i=1}^{G}$ for each sampled output $o^{(i)}$ by running reward function $R(q, o^{(i)})$
  Filter out wrong outputs (i.e., $o^{(i)}$ with $r^{(i)} = 0$) to obtain a dataset $\mathcal{D}_{\text{sft}}$ of correct question-response pairs
  $\pi_\theta \leftarrow \text{SFT}(\pi_\theta, \mathcal{D}_{\text{sft}})$ (Algorithm 1)
end for

Output π_θ

接下来，我们将对 MATH 数据集进行专家迭代。

作为提示，你应该向你的 vLLM 传递一个 `min_tokens` 值，这将确保你不会生成空字符串（这可能会根据你的实现导致下游出现 NaN）。这可以通过以下方式完成

```
sampling_min_tokens = 4
sampling_params = SamplingParams(
      temperature=sampling_temperature,
      max_tokens=sampling_max_tokens,
      min_tokens=sampling_min_tokens,
      n=G,
      seed=seed,
  )
```

与 SFT 类似，你应该使用梯度裁剪，裁剪值为 1.0。

问题 (`expert_iteration_experiment`)：对 MATH 数据集进行专家迭代 (2 点) (6 H100 小时)

对 MATH 数据集（位于 `/data/a5-alignment/MATH/train.jsonl`）进行专家迭代

使用 Qwen 2.5 数学 1.5B 基础模型，调整每个问题的重播次数 G 和 SFT 步骤中使用的训练轮数，并使用 `n_ei_steps` $= 5$。在每个专家迭代步骤中调整批大小（即 $\mathcal{D}_b$ 的大小），设置为 $\{512, 1024, 2048\}$。（您不需要尝试所有这些超参数的组合。只要足够得出每个参数的结论即可。）记录模型在训练过程中响应的熵。确保 vLLM 在第二个答案标签 `</answer>` 处终止生成，就像 SFT 部分所做的那样。

可交付成果：与不同部署配置相关的验证准确率曲线。 尝试在至少 2 种不同的部署次数和轮次数。

可交付成果：一个在 MATH 上验证准确率达到至少 15% 的模型。

可交付成果：一个简短的 2 句话讨论，比较您的 SFT 性能，以及跨 EI 步骤的性能。

可交付成果：模型响应在训练过程中的熵图。

6 政策梯度简介

在语言模型研究中，一个令人兴奋的新发现是：使用经过验证的奖励和强大的基础模型进行强化学习（RL）可以显著提升其推理能力和性能 [OpenAI 等人，2024，DeepSeek-AI 等人， 2025]。目前最强大的开放推理模型，如 DeepSeek R1 和 Kimi k1.5 [团队等人， 2025]，都是使用策略梯度训练的，这是一种强大的强化学习算法，可以优化任意奖励函数。

下面我们简要介绍语言模型上的策略梯度强化学习。我们的讲解紧密基于一些优秀资源，这些资源更深入地探讨了这些概念：OpenAI 的《深度强化学习入门》 [Achiam，2018a]以及 Nathan Lambert 的《人类反馈强化学习（RLHF）》书籍 [Lambert， 2024]。

6.1 语言模型作为策略

一个因果语言模型（LM）的参数 θ 定义了在给定当前文本前缀 $a_t \in \mathcal{V}$ （状态/观测）的情况下下一个标记 s_t 的概率分布。在强化学习的背景下，我们将下一个标记 a_t 视为一个动作，将当前文本前缀 s_t 视为状态。因此，语言模型是一个分类随机策略

$$a_t \sim \pi_\theta(\cdot \mid s_t), \qquad \pi_\theta(a_t \mid s_t) = \left[\mathrm{softmax}\big(f_\theta(s_t)\big)\right]_{a_t}. \tag{3}$$

在用策略梯度优化策略时，需要两种原始操作：

1. 从策略中采样：从上述的离散分布中抽取一个动作 a_t；
2. 评分动作的对数似然：评估 $\log \pi_\theta(a_t \mid s_t)$。

通常，在用 LLM 进行强化学习时， s_t 是到目前为止产生的部分完成/解决方案，而每个 a_t 是解决方案的下一个标记；当发出文本结束标记时（如 `<|end_of_text|>`，或 `</answer>` 在我们 `r1_zero` 提示的情况下）时，回合结束。

6.2 轨迹

一个（有限视野的）轨迹是智能体经历的状态和动作的交错序列：

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T, a_T), \tag{4}$$

T 是轨迹的长度，即， a_T 是文本结束标记或我们已达到最大生成预算（以token计）。

初始状态是根据起始分布绘制的，在带有LLM的强化学习中， $s_0 \sim \rho_0(s_0)$； $\rho_0(s_0)$是一个关于格式化提示的分布。在一般设置中，状态转换遵循某些环境动态 $s_0 \sim \rho_0(s_0)$；。

在带有LLM的强化学习中，环境是确定的：下一个状态是旧的前缀与发出的标记的连接， $s_{t+1} = s_t \| a_t$。轨迹也称为回合或滚动；我们将交替使用这些术语。

6.3 奖励与回报

标量奖励 $r_t = R(s_t, a_t)$ 判断在状态 s_t 下采取行动的即时质量。对于在已验证域上的强化学习，标准做法是将中间步骤的奖励设为零，并将验证奖励分配给最终动作

$$r_T = R(s_T, a_T) := \begin{cases} \text{如果轨迹 } s_T \| a_T \text{ 根据我们的奖励函数与真实轨迹匹配，则为 } 1 \\ 0 \quad \text{否则。} \end{cases}$$

回报 $R(\tau)$ 沿轨迹聚合奖励。两种常见的选择是有限步长无折扣回报

$$R(\tau) := \sum_{t=0}^{T} r_t, \tag{5}$$

和无限步长折扣回报

$$R(\tau) := \sum_{t=0}^{\infty} \gamma^t r_t, \qquad 0 < \gamma < 1. \tag{6}$$

在我们的案例中，我们将使用未折现的公式，因为回合有自然的终止点（文本结束或最大生成长度）。

代理的目标是最大化预期回报

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \right], \tag{7}$$

从而得到优化问题

$$\theta^* = \arg\max_{\theta} J(\theta). \tag{8}$$

6.4 简单策略梯度

接下来，让我们尝试通过预期回报的梯度上升来学习策略参数 θ：

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_k). \tag{9}$$

我们将使用的核心公式是 REINFORCE 策略梯度，如下所示。

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right]. \tag{10}$$

推导策略梯度。我们是如何得到这个公式的？为了完整起见，我们将在此给出这个恒等式的推导过程。我们将用到一些恒等式。

1. 轨迹的概率由下式给出

$$P(\tau \mid \theta) = \rho_0(s_0) \prod_{t=0}^{T} P(s_{t+1} \mid s_t, a_t) \pi_\theta(a_t \mid s_t). \tag{11}$$

因此，轨迹的对数概率为：

$$\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_{t=0}^{T} \big[\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t) \big]. \tag{12}$$

2. 对数导数技巧：

$$\nabla_\theta P = P \nabla_\theta \log P. \tag{13}$$

3. 环境项在 θ 中是常数。 $\rho_0, P(\cdot \mid \cdot)$ 和 $R(\tau)$ 不依赖于策略参数，因此

$$\nabla_\theta \rho_0 = \nabla_\theta P = \nabla_\theta R(\tau) = 0. \tag{14}$$

应用上述事实：

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] && (15)\\
&= \nabla_\theta \sum_\tau P(\tau|\theta) R(\tau) && (16)\\
&= \sum_\tau \nabla_\theta P(\tau|\theta) R(\tau) && (17)\\
&= \sum_\tau P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) \qquad \text{(对数导数技巧)} && (18)\\
&= \mathbb{E}_{\tau \sim \pi_\theta}[\nabla_\theta \log P(\tau|\theta) R(\tau)], && (19)
\end{aligned}$$

因此，代入轨迹的对数概率并利用环境项在 θ 中是常数这一事实，我们得到香草或 REINFORCE 策略梯度：

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)\right]. \tag{20}$$

直观上，这个梯度会增加具有高回报的轨迹中每个动作的对数概率，反之则会降低。

梯度样本估计。 给定一批通过采样收集的 N rollout $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^N$ 一个初始状态 $s_0^{(i)} \sim \rho_0(s_0)$ 以及 n 在环境中运行策略 π_θ ，我们形成了一个无偏的梯度估计器，其表示为

$$\widehat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) R(\tau^{(i)}). \tag{21}$$

该向量用于梯度上升更新 $\theta \leftarrow \theta + \alpha \widehat{g}$。

6.5 策略梯度基线

香草策略梯度的主要问题是梯度估计的高方差。一种常见的缓解方法是从奖励中减去仅依赖于状态的基线函数 b 。这是一种控制变量 [Ross，2022]：其思想是通过减去一个与其相关的项来降低估计器的方差，而不会引入偏差。

让我们定义基线策略梯度：

$$B = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\big(R(\tau) - b(s_t)\big)\right]. \tag{22}$$

例如，一个合理的基线是策略价值函数 $V^\pi(s) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)|s_t = s]$，即如果我们从 $s_t = s$ 开始并遵循从那里开始的策略 π_θ，那么预期回报。然后，量 $(R(\tau) - V^\pi(s_t))$ 直观上就是实际轨迹比预期好多少。

只要基线仅取决于状态，那么基线策略梯度是无偏的。我们可以通过将基线策略梯度重写为以下形式来看到这一点。

$$B = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)\right] - \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)b(s_t)\right]. \tag{23}$$

关注基准项，我们可以看到

$$\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)b(s_t)\right] = \sum_{t=0}^{T}\mathbb{E}_{s_t}\Big[b(s_t)\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]\Big]. \tag{24}$$

一般来说，分数函数的期望值为零： $\mathbb{E}_{x\sim P_\theta}\left[\nabla_\theta \log P_\theta(x)\right] = 0$。因此，式24中的表达式为零，并

$$B = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)\right] - 0 = \nabla_\theta J(\pi_\theta), \tag{25}$$

所以我们得出基准策略梯度是无偏的。稍后我们将进行实验，看看基准化是否提高了下游性能。

关于策略梯度“损失”的说明。当我们在PyTorch等框架中实现策略梯度方法时，我们将定义一个所谓的策略梯度损失 `pg_loss` ，调用 `pg_loss.backward()` 将用我们的近似策略梯度 $\widehat{g}$ 填充模型参数的梯度缓冲区。在数学上，它可以表述为

$$\texttt{pg_loss} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\log \pi_\theta(a_t^{(i)}|s_t^{(i)})(R(\tau^{(i)}) - b(s_t^{(i)})). \tag{26}$$

`pg_loss` 不是规范意义上的损失——在训练集或验证集上报告 `pg_loss` 作为评估指标没有意义，良好的验证 `pg_loss` 并不表明我们的模型泛化得很好。 `pg_loss` 真正只是一个标量，当我们调用 `pg_loss.backward()`时，通过反向传播获得的梯度就是近似策略梯度 $\widehat{g}$。

进行强化学习时，你应当始终记录和报告训练与验证奖励。这些是“有意义的”评估指标，也是我们通过策略梯度方法试图优化的目标。

6.6 离线策略梯度

REINFORCE 是一种在线策略算法：训练数据由我们正在优化的同一策略收集。要理解这一点，让我们写出 REINFORCE 算法：

1. 从当前策略 $\{\tau^{(i)}\}_{i=}^{N}$ 中采样一批 rollout $\{\tau^{(i)}\}_{i=}^{N}$ 1 从当前政策 π_θ。
2. 近似策略梯度为 $\nabla_\theta J(\pi_\theta) \approx \widehat{g} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})R(\tau^{(i)})$.
3. 使用计算出的梯度更新策略参数： $\theta \leftarrow \theta + \alpha\widehat{g}$。

我们需要进行大量推理来采样一批新的 rollout，但最终只进行一次梯度步进。语言模型的特性通常无法在单步内发生显著变化，因此这种在线策略方法效率极低。

离线策略梯度。在离线学习中，我们采样的是来自某个与我们正在优化的策略不同的策略的 rollout。像 PPO 和 GRPO 这样流行策略梯度算法的离线变种，使用策略 $\pi_{\theta_{\text{old}}}$ 的先前版本的 rollout 来优化当前策略 π_θ。离线策略梯度估计是

$$\widehat{g}_{\text{off-policy}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\frac{\pi_\theta(a_t^{(i)}|s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)}|s_t^{(i)})}\nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})R(\tau^{(i)}). \tag{27}$$

这看起来像是香草策略梯度的重要性采样版本，带有重加权项 $\frac{\pi_\theta(a_t^{(i)}|s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)}|s_t^{(i)})}$。确实，公式 27 可以通过重要性采样并通过一个近似方法推导出来，只要 π_θ 和 $\pi_{\theta_{\text{old}}}$ 不会相差太大：参见 Degris 等人 [2013] 的更多讨论。

7 组相对策略优化

接下来，我们将描述组相对策略优化（GRPO），这是您将要实现和用于解决数学问题的策略梯度变体。

7.1 GRPO 算法

优势估计。GRPO 的核心思想是从策略 π_θ 对每个问题采样多个输出，并使用它们来计算一个基线。这很方便，因为我们避免了学习神经值函数 $V_\phi(s)$ 的需要，而神经值函数可能很难训练，并且从系统角度来看很繁琐。对于一个问题 q 和组输出 $\{o^{(i)}\}_{i=1}^{G} \sim \pi_\theta(\cdot|q)$，设 $r^{(i)} = R(q, o^{(i)})$ 是第 i 个输出的奖励。DeepSeekMath [Shao 等人 2024] 和 DeepSeek R1 [DeepSeek-AI 等人 2025] 将第 i 个输出的组归一化奖励计算为

$$A^{(i)} = \frac{r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \ldots, r^{(G)})}{\text{std}(r^{(1)}, r^{(2)}, \ldots, r^{(G)}) + \texttt{advantage_eps}}, \tag{28}$$

`advantage_eps` 是一个小的常数，用于防止除以零。请注意，这个优势 $A^{(i)}$ 对响应中的每个标记都是相同的，即 $A_t^{(i)} = A^{(i)}\ \forall t \in 1 \ldots |o^{(i)}|$、$A_t^{(i)} = A^{(i)}\ \forall t \in 1 \ldots |o^{(i)}|$、$A_t^{(i)} = A^{(i)}\ \forall t \in 1 \ldots |o^{(i)}|$，因此我们在后续内容中省略 t 下标。

高层算法。在深入GRPO目标之前，让我们先通过写出Shao等人 [2024] 在算法3中的算法，来了解训练循环的概念。[2]

GRPO目标。GRPO目标结合了三个思想：

1. 离策略策略梯度，如公式27所示。
2. 使用组归一化计算优势 $A^{(i)}$，如公式28所示。
3. 剪辑机制，如近端策略优化（PPO，Schulman等人 [2017]）。

裁剪的目的是在单批次多个梯度步长时保持稳定性。它通过确保策略 π_θ 不偏离旧策略太远来实现。

[2]这是DeepSeekMath的GRPO的一种特殊情况，具有经过验证的奖励函数，没有KL项，且参考模型和奖励模型没有迭代更新。

Algorithm 3 Group Relative Policy Optimization (GRPO)

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward function R; task questions $\mathcal{D}$

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for step = 1, ..., `n_grpo_steps` **do**
  Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$
  Set the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
  Sample G outputs $\{o^{(i)}\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
  Compute rewards $\{r^{(i)}\}_{i=1}^{G}$ for each sampled output $o^{(i)}$ by running reward function $R(q, o^{(i)})$
  Compute $A^{(i)}$ with group normalization (Eq. 28)
  for train step = 1, ..., `n_train_steps_per_rollout_batch` **do**
    Update the policy model π_θ by maximizing the GRPO-Clip objective (to be discussed, Eq. 29)
  end for
end for

Output π_θ

让我们首先写出完整的 GRPO-Clip 目标，然后我们可以对裁剪的作用建立一些直观理解：

$$J_{\text{GRPO-Clip}}(\theta) = \mathbb{E}_{q\sim\mathcal{D},\ \{o^{(i)}\}_{i=1}^{G}\sim\pi_\theta(\cdot|q)} \left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o^{(i)}|}\sum_{t=1}^{|o^{(i)}|}\underbrace{\min\left(\frac{\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})}A^{(i)}, \text{clip}\left(\frac{\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})}, 1-\epsilon,\ 1+\epsilon\right)A^{(i)}\right)}_{\text{per-token objective}}\right]. \tag{29}$$

超参数 $\epsilon > 0$ 控制策略可以变化多少。要理解这一点，我们可以按照 Achiam [2018a,b] 的方式，以更直观的方式重写每个 token 的目标函数。

$$g(\epsilon, A^{(i)}) = \begin{cases}(1+\epsilon)A^{(i)} & \text{if } A^{(i)} \geq 0\\ (1-\epsilon)A^{(i)} & \text{if } A^{(i)} < 0.\end{cases} \tag{30}$$

我们可以将每个 token 的目标函数重写为

$$\text{per-token objective} = \min\left(\frac{\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})}A^{(i)}, g(\epsilon, A^{(i)})\right)$$

现在我们可以分情况推理。当优势 $A^{(i)}$ 为正时，每个 token 的目标函数简化为

$$\text{per-token objective} = \min\left(\frac{\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})}, 1+\epsilon\right)A^{(i)}.$$

由于 $A^{(i)} > 0$，如果动作 $o_t^{(i)}$ 在 π_θ 下变得更可能，即 $\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})$ 增加时，目标函数会上升。截断与最小值一起限制了目标函数可以增加的最大幅度：一旦 $\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)}) > (1+\epsilon)\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})$，这个每个 token 的目标函数就会达到其最大值 $(1+\epsilon)A^{(i)}$。因此，策略 π_θ 没有动力偏离旧策略 $\pi_{\theta_{\text{old}}}$ 太远。

类似地，当优势 $A^{(i)}$ 为负时，模型试图将 $\pi_\theta(o_t^{(i)} \mid q, o_{<t}^{(i)})$ 推低，但不会被激励将其降低到 $(1-\epsilon)\pi_{\theta_{\text{old}}}(o_t^{(i)} \mid q, o_{<t}^{(i)})$ 以下（完整论证参见 Achiam [2018b]）。

7.2 实现

现在我们已经对GRPO训练循环和目标有了高级别的理解，我们将开始实现其中的部分内容。SFT和EI部分中实现的许多内容也将被用于GRPO。

计算优势（组归一化奖励）。首先，我们将实现计算每个展开批次中每个示例的优势的逻辑，即组归一化奖励。我们将考虑两种获取组归一化奖励的可能方法：上述公式28中提出的方法，以及一种最近简化的方法。

GRPO [刘博士等， 2025] 指出，通过 $\text{std}(r^{(1)}, r^{(2)}, \ldots, r^{(G)})$ 对一批答案正确性差异较小的奖励问题进行归一化，可能并非理想选择。他们建议直接移除归一化步骤，计算

$$A^{(i)} = r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \ldots, r^{(G)}). \tag{31}$$

我们将实现这两种变体，并在作业的后面比较它们的性能。

Problem (`compute_group_normalized_rewards`): Group normalization (2 points)

Deliverable: Implement a method `compute_group_normalized_rewards` that calculates raw rewards for each rollout response, normalizes them within their groups, and returns both the normalized and raw rewards along with any metadata you think is useful.
The following interface is recommended:

```python
def compute_group_normalized_rewards(
    reward_fn,
    rollout_responses,
    repeated_ground_truths,
    group_size,
    advantage_eps,
    normalize_by_std,
):
```

Compute rewards for each group of rollout responses, normalized by the group size.

Args:

`reward_fn: Callable[[str, str], dict[str, float]]` Scores the rollout responses against the ground truths, producing a dict with keys `"reward"`, `"format_reward"`, and `"answer_reward"`.

`rollout_responses: list[str]` Rollouts from the policy. The length of this list is `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.

`repeated_ground_truths: list[str]` The ground truths for the examples. The length of this list is `rollout_batch_size`, because the ground truth for each example is repeated `group_size` times.

`group_size: int` Number of responses per question (group).

`advantage_eps: float` Small constant to avoid division by zero in normalization.

`normalize_by_std: bool` If `True`, divide by the per-group standard deviation; otherwise subtract only the group mean.

Returns:

`tuple[torch.Tensor, torch.Tensor, dict[str, float]]`.

`advantages` shape `(rollout_batch_size,)`. Group-normalized rewards for each rollout response.

raw_rewards 形状 (rollout_batch_size,)。每个rollout的非标准化奖励响应。

metadata 您可以选择其他要记录的统计数据（例如均值、标准差、奖励的最大/最小值）。

要测试您的代码，请实现 [adapters.run_compute_group_normalized_rewards]。然后，使用 `uv run pytest -k test_compute_group_normalized_rewards` 运行测试，并确保您的实现通过它。

朴素策略梯度损失。接下来，我们将实现一些用于计算“损失”的方法。

作为提醒/免责声明，这些在规范意义上并非真正的损失，不应作为评估指标报告。在强化学习（RL）中，您应该跟踪训练和验证回报以及其他指标（参见第6.5节进行讨论）。

我们将从朴素策略梯度损失开始，它简单地通过动作的对数概率（并取负值）乘以优势。对于问题 q、响应 o 和响应标记 o_t，朴素按标记的策略梯度损失是

$$-A_t \cdot \log p_\theta(o_t|q, o_{<t}). \tag{32}$$

问题 (compute_naive_policy_gradient_loss): 傻瓜策略梯度 **(1 point)**

可交付成果：实现一个方法 `compute_naive_policy_gradient_loss`，用于计算按标记计算策略梯度损失，使用原始奖励或预计算的优势。
推荐以下接口：

```
def compute_naive_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
) -> torch.Tensor:
```

在每个标记处计算策略梯度损失，其中 `raw_rewards_or_advantages` 要么是原始奖励或已归一化的优势。

参数：

raw_rewards_or_advantages: torch.Tensor 形状 (batch_size, 1)，标量每个回滚响应的奖励/优势。

policy_log_probs: torch.Tensor 形状 (batch_size, sequence_length)，对数概率每个标记。

返回：

torch.Tensor 形状 (batch_size, sequence_length)，即每个token的政策梯度损失（将在训练循环中跨批次和序列维度进行聚合）。

实现技巧：

- 在 `raw_rewards_or_advantages` 维度上进行广播。

要测试你的代码，请实现[adapters.run_compute_naive_policy_gradient_loss]。然后运行 `uv run pytest -k test_compute_naive_policy_gradient_loss` 并确保测试通过。

GRPO-Clip 损失。 接下来，我们将实现更有趣的 GRPO-Clip 损失。

每个token的GRPO-Clip损失是

$$-\min\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q,o_{<t})}A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q,o_{<t})}, 1-\epsilon, 1+\epsilon\right)A_t\right). \tag{33}$$

Problem (`compute_grpo_clip_loss`): GRPO-Clip loss (2 points)

Deliverable: Implement a method `compute_grpo_clip_loss` that computes the per-token GRPO-Clip loss.
The following interface is recommended:

```python
def compute_grpo_clip_loss(
    advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    old_log_probs: torch.Tensor,
    cliprange: float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Args:

`advantages: torch.Tensor` Shape `(batch_size, 1)`, per-example advantages A.

`policy_log_probs: torch.Tensor` Shape `(batch_size, sequence_length)`, per-token log probs from the policy being trained.

`old_log_probs: torch.Tensor` Shape `(batch_size, sequence_length)`, per-token log probs from the old policy.

`cliprange: float` Clip parameter ϵ (e.g. 0.2).

Returns:

`tuple[torch.Tensor, dict[str, torch.Tensor]]`.

loss `torch.Tensor` of shape `(batch_size, sequence_length)`, the per-token clipped loss.

metadata dict containing whatever you want to log. We suggest logging whether each token was clipped or not, i.e., whether the clipped policy gradient loss on the RHS of the min was lower than the LHS.

Implementation tips:

- Broadcast `advantages` over `sequence_length`.

To test your code, implement `[adapters.run_compute_grpo_clip_loss]`. Then run `uv run pytest -k test_compute_grpo_clip_loss` and ensure the test passes.

策略梯度损失包装器。我们将运行消融实验，比较三种不同的策略梯度版本：

(a) `no_baseline`: 无基准的朴素策略梯度损失，即优势只是原始奖励$A = R(q, o)$。

(b) `reinforce_with_baseline`: 使用我们组归一化的奖励作为优势的朴素策略梯度损失。如果 $\bar{r}$ 是来自 `compute_group_normalized_rewards` 的组归一化奖励（这些奖励可能或不可能由组标准差归一化），那么 $A = \bar{r}$。

(c) `grpo_clip`: GRPO-Clip 损失。

为了方便，我们将实现一个包装器，以便我们能够轻松地在这三种策略梯度之间切换损失。

Problem (`compute_policy_gradient_loss`): Policy-gradient wrapper (1 point)

可交付成果：实现 `compute_policy_gradient_loss`，一个便捷的包装器，它将分发到正确的损失例程 (`no_baseline`, `reinforce_with_baseline` 或 `grpo_clip`) 并返回每个标记的损失以及任何辅助统计信息。

以下界面推荐：

```python
def compute_policy_gradient_loss(
    policy_log_probs: torch.Tensor,
    loss_type: Literal["no_baseline", "reinforce_with_baseline", "grpo_clip"],
    raw_rewards: torch.Tensor | None = None,
    advantages: torch.Tensor | None = None,
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

选择并计算所需的策略梯度损失。

Args:

`policy_log_probs` `(batch_size, sequence_length)`, per-token log-probabilities from the policy being trained.

`loss_type` One of `"no_baseline"`, `"reinforce_with_baseline"`, or `"grpo_clip"`.

`raw_rewards` Required if `loss_type == "no_baseline"`; shape `(batch_size, 1)`.

`advantages` Required for `"reinforce_with_baseline"` and `"grpo_clip"`; shape `(batch_size, 1)`.

`old_log_probs` Required for `"grpo_clip"`; shape `(batch_size, sequence_length)`.

`cliprange` Required for `"grpo_clip"`; scalar ϵ used for clipping.

返回：

`tuple[torch.Tensor, dict[str, torch.Tensor]]`.

loss `(batch_size, sequence_length)`, per-token loss.
元数据字典，来自底层例程的统计信息（例如GRPO-Clip的裁剪比例）。

实现技巧：

- Delegate to `compute_naive_policy_gradient_loss` or `compute_grpo_clip_loss`.
- Perform argument checks (see assertion pattern above).
- Aggregate any returned metadata into a single dict.

要测试你的代码，请实现`[adapters.run_compute_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss` 并验证它是否通过。

掩码均值。到目前为止，我们已经有了计算优势、日志概率、每个token的损失以及诸如每个token的熵和剪裁分数等有用统计数据的逻辑。为了将形状为 `(batch_size, sequence_length)` 的每个token损失张量减少为一个损失向量（每个示例一个标量），我们

将计算损失在序列维度上的均值，但仅针对响应对应的索引（即，满足 `mask[i, j]==1`的token位置）。

通过序列长度进行归一化在大多数使用LLM进行RL的代码库中已成为规范，但并不明显这是正确的方法——你可能会注意到，查看(21)中策略梯度估计的陈述时，没有归一化因子 $\frac{1}{T^{(i)}}$。我们将从这种标准原语开始，通常称为 `masked_mean`，但稍后会测试我们SFT期间实现的 `masked_normalize` 方法。

我们将允许指定计算均值所用的维度，如果 `dim` 是 `None`，我们将计算所有掩码元素上的均值。这有助于获取响应标记上的平均每标记熵、剪裁分数等。

Problem (`masked_mean`): Masked mean (1 point)

Deliverable: Implement a method `masked_mean` that averages tensor elements while respecting a boolean mask.
The following interface is recommended:

```
def masked_mean(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    dim: int | None = None,
) -> torch.Tensor:
```

Compute the mean of `tensor` along a given dimension, considering only those elements where `mask == 1`.

Args:

`tensor: torch.Tensor` The data to be averaged.

`mask: torch.Tensor` Same shape as `tensor`; positions with `1` are included in the mean.

`dim: int | None` Dimension over which to average. If `None`, compute the mean over all masked elements.

Returns:

`torch.Tensor` The masked mean; shape matches `tensor.mean(dim)` semantics.

To test your code, implement `[adapters.run_masked_mean]`. Then run `uv run pytest -k test_masked_mean` and ensure it passes.

GRPO微批次训练步骤。现在我们准备好为GRPO实现单个微批次训练步骤（回想一下，对于训练小批次，如果 `gradient_accumulation_steps` >1，我们会迭代许多微批次）。

具体来说，给定原始奖励或优势以及日志概率，我们将计算每标记损失，使用`masked_mean` 聚合成每个示例的标量损失，对批次维度求平均，调整梯度累积，并进行反向传播。

问题 (`grpo_microbatch_train_step`)：微批处理训练步骤 (3 points)

可交付成果：为GRPO实现单个微批处理更新，包括策略梯度损失、带掩码的均值计算和梯度缩放。

以下界面建议：

```python
def grpo_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    loss_type: Literal["no_baseline", "reinforce_with_baseline", "grpo_clip"],
    raw_rewards: torch.Tensor | None = None,
    advantages: torch.Tensor | None = None,
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

对微批处理执行前向和后向传递。

Args:

policy_log_probs (batch_size, sequence_length)，来自正在训练的策略的每个token的日志概率。

response_mask (batch_size, sequence_length)，1表示响应token，0表示prompt/padding.

gradient_accumulation_steps Number of microbatches per optimizer step.

loss_type One of "no_baseline", "reinforce_with_baseline", "grpo_clip".

raw_rewards Needed when loss_type == "no_baseline"; shape (batch_size, 1).

advantages Needed when loss_type != "no_baseline"; shape (batch_size, 1).

old_log_probs Required for GRPO-Clip; shape (batch_size, sequence_length).

cliprange Clip parameter ϵ for GRPO-Clip.

返回：

tuple[torch.Tensor, dict[str, torch.Tensor]].

损失标量张量。微批处理的损失，已调整为梯度累积。我们返回此值以便记录。

元数据Dict，包含底层损失调用产生的元数据，以及你可能想要记录的任何其他统计信息。

实现技巧：

- 你应该在这个函数中调用 `loss.backward()` 。确保针对渐变进行调整累积。

要测试你的代码，实现 [adapters.run_grpo_microbatch_train_step]。然后运行 `uv run pytest -k test_grpo_microbatch_train_step` 并确认它通过。

将所有内容整合起来：GRPO 训练循环。现在我们将整合一个完整的 GRPO 训练循环。你应该参考第 7.1 节中的算法，以获取整体结构，并在适当的地方使用我们实现的方法。

下面我们提供一些初始超参数。如果你有正确的实现，使用这些参数应该能看到合理的结果。

```python
n_grpo_steps: int = 200
learning_rate: float = 1e-5
```

```
advantage_eps: float = 1e-6
rollout_batch_size: int = 256
group_size: int = 8
sampling_temperature: float = 1.0
sampling_min_tokens: int = 4  # As in Expiter, disallow empty string responses
sampling_max_tokens: int = 1024
epochs_per_rollout_batch: int = 1  # On-policy
train_batch_size: int = 256  # On-policy
gradient_accumulation_steps: int = 128  # microbatch size is 2, will fit on H100
gpu_memory_utilization: float = 0.85
loss_type: Literal[
    "no_baseline",
    "reinforce_with_baseline",
    "grpo_clip",
] = "reinforce_with_baseline"
use_std_normalization: bool = True
optimizer = torch.optim.AdamW(
    policy.parameters(),
    lr=learning_rate,
    weight_decay=0.0,
    betas=(0.9, 0.95),
)
```

这些默认超参数会把你置于在线策略设置中——对于每个展开批次，我们执行一次梯度步。在超参数方面，这意味着 `train_batch_size` 等于 `rollout_batch_size`，而 `epochs_per_rollout_batch` 等于 1。

这里有一些用于检查合理性的断言和常量，它们应该能消除一些边缘情况并引导你走向正确的方向：

```
assert train_batch_size % gradient_accumulation_steps == 0, (
    "train_batch_size must be divisible by gradient_accumulation_steps"
)
micro_train_batch_size = train_batch_size // gradient_accumulation_steps
assert rollout_batch_size % group_size == 0, (
    "rollout_batch_size must be divisible by group_size"
)
n_prompts_per_rollout_batch = rollout_batch_size // group_size
assert train_batch_size >= group_size, (
    "train_batch_size must be greater than or equal to group_size"
)
n_microbatches_per_rollout_batch = rollout_batch_size // micro_train_batch_size
```

这里还有一些额外的提示：

- 请记得使用 `r1_zero` 提示，并让 vLLM 在第二个答案标签 `</answer>` 处停止生成，就像之前的实验一样。
- 我们建议使用 `typer` 进行参数解析。
- 使用梯度裁剪，裁剪值为 1.0。
- 你应该定期记录验证奖励（例如，每5或10步记录一次）。你应该至少评估一次，并重复使用它们每个epoch。使用 1024 个验证示例来比较超参数，因为 CoT/RL 评估可能会有噪声。
- 通过我们的损失实现，GRPO-Clip 仅应在离线策略（因为它需要旧的日志概率）时使用。
- 在离线策略设置中，每个回滚批次进行多个梯度更新时，重新计算每个时期的旧日志概率将是浪费的。相反，我们可以计算旧的日志概率

一次，并在每个epoch中重复使用它们。
- 你不应该对旧的日志概率进行区分。
- 对于每个优化器更新，你应该记录以下一项或多项：
  - 损失。
  - 梯度范数。
  - 令牌熵。
  - 如果是离线策略，则记录剪裁分数。
  - 训练奖励（总数、格式和答案）。
  - 你认为对调试有用的任何其他内容。

问题 (`grpo_train_loop`)：GRPO 训练循环 (5 分)

交付物：实现一个完整的 GRPO 训练循环。开始在 MATH 上训练策略，并确认验证奖励在提升，同时时间步长上的策略展开也合理。提供一张显示验证奖励随步长变化的图表，以及几个时间步长上的策略展开示例。

8 个 GRPO 实验

现在我们可以开始实验我们的 GRPO 训练循环，尝试不同的超参数和算法调整。每个实验将需要 2 个 GPU，一个用于 vLLM 实例，另一个用于策略。

关于提前停止运行的说明。如果在 200 个 GRPO 步骤之前（例如，某个配置出现发散或明显次优），你看到超参数之间存在显著差异，那么当然可以随时提前停止实验，为后续运行节省时间和计算资源。下面提到的 GPU 小时数是一个粗略估计。

问题 (`grpo_learning_rate`)：调整学习率 (2 分) (6 H100 小时)

从上述建议的超参数开始，对学习率进行扫描，并报告最终的验证答案奖励（如果优化器发散，请注明发散）。

交付物：与多个学习率相关的验证奖励曲线。
可交付成果：一个在MATH上验证准确率至少达到25%的模型。
可交付成果：关于其他已记录指标上你注意到的任何其他趋势的简短讨论（2句话）。

对于其余的实验，你可以使用你在上面扫描中表现最佳的 learning rate。

基线的影响。继续使用上述超参数（除了你调整的 learning rate），我们将现在研究基线的影响。我们处于 on-policy 设置中，因此我们将比较损失类型：

- `no_baseline`
- `reinforce_with_baseline`

请注意， `use_std_normalization` 在默认超参数中是 `True`。

问题 (`grpo_baselines`)：基线的影响 (2分) (2 H100小时)

使用 `reinforce_with_baseline` 和 `no_baseline`训练策略。
交付成果：与每种损失类型相关的验证奖励曲线。
交付成果：关于你在其他记录指标上发现的任何其他趋势的简短讨论（2句话）。

接下来的几次实验中，你应该使用上述实验中找到的最佳损失类型。

长度归一化。正如我们在实现 `masked_mean` 时所暗示的那样，对损失进行平均并不是必要的，甚至可能是错误的。如何对损失进行求和是一个重要的超参数，它会导致对策略动作的信用分配产生不同类型的结果。

让我们通过一个来自 Lambert [2024] 的例子来说明这一点。检查 GRPO 训练步骤，我们首先通过忽略剪裁来获得每个 token 的策略梯度损失：

```
advantages  # (batch_size, 1)
per_token_probability_ratios  # (batch_size, sequence_length)
per_token_loss = -advantages * per_token_probability_ratios
```

我们在序列长度上广播了优势。让我们比较两种聚合每个token损失的方法：

- 我们实现的 `masked_mean`，它对每个序列中未掩码的token进行平均。
- 对每个序列中未掩码的 token 求和，然后除以一个常数标量（我们的 `masked_normalize` 方法支持与 `constant_normalizer != 1.0`）[刘等人，2025年，余等人，2025]

我们将考虑一个例子，其中批次大小为 2，第一个响应有 4 个 token，第二个响应有 7 个 token。然后，我们可以看到这些归一化方法如何影响梯度。

```
from your_utils import masked_mean, masked_normalize

ratio = torch.tensor([
    [1, 1, 1, 1, 1, 1, 1,],
    [1, 1, 1, 1, 1, 1, 1,],
], requires_grad=True)

advs = torch.tensor([
    [2, 2, 2, 2, 2, 2, 2,],
    [2, 2, 2, 2, 2, 2, 2,],
])

masks = torch.tensor([
    # generation 1: 4 tokens
    [1, 1, 1, 1, 0, 0, 0,],
    # generation 2: 7 tokens
    [1, 1, 1, 1, 1, 1, 1,],
])

# Normalize with each approach
max_gen_len = 7
masked_mean_result = masked_mean(ratio * advs, masks, dim=1)
masked_normalize_result = masked_normalize(
    ratio * advs, masks, dim=1, constant_normalizer=max_gen_len)

print("masked_mean", masked_mean_result)
print("masked_normalize", masked_normalize_result)

# masked_mean tensor([2., 2.], grad_fn=<DivBackward0>)
# masked_normalize tensor([1.1429, 2.0000], grad_fn=<DivBackward0>)

masked_mean_result.mean().backward()
print("ratio.grad", ratio.grad)
# ratio.grad:
```

```
# tensor([[0.2500, 0.2500, 0.2500, 0.2500, 0.0000, 0.0000, 0.0000],
#         [0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429]])
ratio.grad.zero_()

masked_normalize_result.mean().backward()
print("ratio.grad", ratio.grad)
# ratio.grad:
# tensor([[0.1429, 0.1429, 0.1429, 0.1429, 0.0000, 0.0000, 0.0000]
#         ,[0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429]])
```

问题 (think_about_length_normalization): 考虑长度归一化 (1 点)

可交付成果：比较两种方法（目前尚未运行实验）。每种方法有哪些优缺点？在哪些特定设置或示例中，一种方法似乎更好？

现在，让我们通过实证来比较 `masked_mean` 和 `masked_normalize`。

问题 (grpo_length_normalization)：长度归一化的影响（2 分）(2 H100 hrs)

交付物：比较 `masked_mean` 和 `masked_normalize` 的归一化效果，并进行端到端的 GRPO 训练运行。报告验证答案奖励曲线。评论发现结果，
包括任何有明显趋势的其他指标。
提示：考虑与稳定性相关的指标，例如梯度范数。

针对性能更好的长度归一化方法，为后续实验进行修正。

使用组标准差进行归一化。回想一下我们的标准实现 `compute_group_normalized_rewards`（基于 Shao 等人 [2024], DeepSeek-AI 等人 [2025]），我们在其中通过组标准差进行归一化。Liu 等人 [2025] 指出，除以组标准差可能会向训练过程引入不必要的偏差：标准差较低的问题（例如，所有奖励几乎都是 1 或 0 的太简单或太难的问题）在训练期间会获得更高的权重。Liu 等人 [2025] 提出移除通过标准差进行归一化，我们已经在
implemented in `compute_group_normalized_rewards` and will now test.

问题 (grpo_group_standard_deviation)：标准差归一化的影响 (2 点) (2 H100 小时)

交付物：比较 `use_std_normalization == True` 和 `use_std_normalization == False` 的性能。报告验证答案奖励曲线。评论发现结果，包括任何有明显趋势的其他指标。

提示：考虑与稳定性相关的指标，例如梯度范数。

针对以下实验，修复到性能更好的组归一化方法。

离策略与在线策略。我们迄今为止实验过的超参数都是在线策略：我们对每个回放批次只进行一次梯度步进，因此我们几乎完全使用的是“原则性”

策略梯度近似 $\hat{g}$ （除了上述提到的长度和优势归一化选择）。

虽然这种方法在理论上是有道理且稳定的，但它效率低下。策略的生成需要缓慢的输出，因此成为 GRPO 的主要成本；每次展开批次只进行一次梯度步长似乎很浪费，可能不足以显著改变策略的行为。

我们现在将尝试离线策略训练，其中我们为每个展开批次进行多个梯度步长（甚至多个 epoch）。

问题 (`grpo_off_policy`)：实现离线策略 GRPO

交付物：实现离线策略 GRPO 训练。

根据您上面完整的 GRPO 训练循环的实现，您可能已经拥有执行此操作的基础设施。如果没有，您需要实现以下内容：

- 您应该能够为每个展开批次进行多个 epoch 的梯度步长，其中每个展开批次的 epoch 数量和优化器更新次数由 `rollout_batch_size`、 `epochs_per_rollout_batch` 和 `train_batch_size` 控制。
- 请修改你的主训练循环，在每次策略的 rollout 批量生成阶段后、梯度步的内部循环之前获取响应的 logprobs，这些将是 `old_log_probs`。我们建议使用 `torch.inference_mode()`。
- 你应该使用 `"GRPO-Clip"` 损失类型。

现在我们可以使用每个 rollout 批量的训练轮数和优化器更新次数来控制我们离策略的程度。

问题 (`grpo_off_policy_sweep`)：离策略 GRPO 超参数扫描 (4 分) (12 H100 小时)

可交付成果：修复 `rollout_batch_size = 256`，选择一个范围覆盖 `epochs_per_rollout_batch` 和 `train_batch_size` 进行扫描。首先进行一次广泛的扫描，使用有限的 GRPO 步数 (<50) 以了解性能概况，然后进行更聚焦的扫描，使用更多的 GRPO 步数（200）。提供一份简要的实验日志，说明你选择的范围。

与使用 `epochs_per_rollout_batch = 1` 和 `train_batch_size` =256 的在线策略运行进行比较，报告关于验证步数以及关于实际运行时间的图表。

报告验证答案奖励曲线。评论你的发现，包括任何有明显趋势的其他指标，例如熵和响应长度。比较模型在训练期间响应的熵与你在 EI 实验中观察到的结果。

提示：你需要更改 `gradient_accumulation_steps` 以保持内存使用量恒定。

在离策略设置中移除裁剪。回想一下，GRPO-Clip中裁剪的目的是防止策略在单个回滚批次上进行多次梯度步长时偏离旧策略太远。接下来，我们将在离策略设置中移除裁剪，以测试其实际上有多必要。换句话说，我们将使用按token的损失

$$-\frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t})} A_t. \tag{34}$$

问题 (grpo_off_policy_clip_ablation): 离策略GRPO-Clip移除 (2分) (2 H100小时)

可交付成果：将未裁剪的每token损失实现为一种新的损失类型 "GRPO-No-Clip"。使用上一问题中表现最佳的离线策略超参数，运行未裁剪版本的损失。报告验证答案奖励曲线。与你的 GRPO-Clip 运行结果相比，评论发现，包括熵、响应长度和梯度范数等有明显趋势的其他指标。

提示的影响。作为最后一次消融实验，我们将研究一个令人惊讶的现象：RL过程中使用的提示对模型性能有巨大影响，具体取决于模型的预训练方式。我们不会在 cs336_alignment/prompts/r1_zero.prompt 使用 R1-Zero 提示，而是使用一个极其简单的提示 cs336_alignment/prompts/question_only.prompt：

```
{question}
```

你将使用此提示进行训练和验证，并将你的奖励函数（用于训练和验证）更改为位于 cs336_alignment/drgrpo_grader.py 的 question_only_reward_fn 。

问题 (grpo_prompt_ablation)：提示词消融（2分）(2 H100小时)

交付物：报告R1-Zero提示词和仅问题提示词的验证答案奖励曲线。指标如何比较，包括任何有明显趋势的其他指标（如熵、响应长度和梯度范数）？尝试解释你的发现。

9 排行榜：GRPO on MATH

作为（强制）作业的最后一部分，你将实验使用方法，在2 H100 GPU上4小时的训练时间内获得尽可能高的验证奖励。

模型。我们将继续使用Qwen 2.5 Math 1.5B Base模型。

数据集。我们将继续使用集群上位于/data/a5-alignment/MATH/train.jsonl 和 /data/a5-alignment/MATH/validation.jsonl的MATH训练和验证数据集。你不允许使用任何其他数据或在更强的模型的推理链上进行SFT。你必须报告整个验证集（所有5K个示例）的验证准确率，使用上述采样超参数（温度1.0，最大token 1024）。你允许过滤训练集，或设计数据上的课程，如你所需。你必须使用R1-Zero提示词进行验证，在验证期间，你必须使用starter code中提供的恰好 r1_zero_reward_fn 奖励函数（如果你愿意，你允许开发另一个用于训练的奖励函数）。

算法。你可以自由调整超参数或完全更换训练算法，只要你不使用任何额外数据或另一个模型（如果你想，可以自由使用更多模型副本）。

系统优化。你可能会注意到，在我们上面简单的 GRPO 实现中，至少有一个 GPU 始终处于空闲状态。通过改进我们管道的系统特性，你很可能会发现显著的改进。例如，你可以考虑为 rollout 或训练使用较低精度， `torch.compile`，以及其他系统优化。你绝对不受限制于将 vLLM 放在一个设备上，而将训练策略放在另一个设备上，并且鼓励你思考更好的并行化方式。

想法。关于可能的改进建议，请参阅以下仓库：

- veRL
- trl
- torchtune
- oat

关于KL散度。我们还注意到，在上述实验中，我们没有包含相对于某个参考模型的KL散度项（通常这是一个冻结的SFT或预训练检查点）。在我们的实验以及其他文献中的研究 [刘等人，2025，NTT123, 2025]，我们发现省略KL项对性能没有影响，同时节省了GPU内存（无需存储参考模型）。然而，许多GRPO仓库默认包含它，我们鼓励您尝试KL或其他形式的正则化，只要您使用Qwen 2.5 Math 1.5B Base或通过您的算法获得的某些模型即可。

问题 (`leaderboard`)：排行榜 (16 points) (16 H100 hrs)

交付物：报告在 2 块 H100 GPU 上训练 4 小时内获得的验证准确率，以及一张显示验证准确率与墙上时间关系的截图，其中 x 轴结束于 ≤ 4 小时。提醒一下，我们对你的评估有以下约束：

1. 你的验证准确率应为整个 MATH 验证集（所有 5K 个样本）的平均准确率。
2. 你必须在验证时使用 R1-Zero 提示。
3. 你必须使用温度 1.0 和最大 token 1024，通过 vLLM 进行评估。
4. 你必须通过平均 starter code 中提供的 `r1_zero_reward_fn` 奖励函数生成的答案奖励来计算验证准确率。

第10章 尾声

恭喜你完成了课程的最后一项作业！你应该为自己的努力感到骄傲。我们希望你在从零开始构建现代语言模型的主要组件的过程中，享受了学习其基础原理的乐趣。

参考文献

丹·亨德里克斯、科林·伯恩斯、萨鲁夫·卡达瓦斯、阿库尔·阿罗拉、史蒂文·巴萨尔特、埃里克·唐、道恩·宋和雅各布·斯坦哈特。使用数学数据集衡量数学问题解决能力，2021年。URL `https://arxiv.org/abs/2103.03874`。

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. URL https://arxiv.org/abs/2112.00114.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022. URL https://arxiv.org/abs/2203.14465.

Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search, 2017. URL https://arxiv.org/abs/1705.08439.

OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong,

伊丽莎白·普罗厄尔，伊恩·张，埃里克·米切尔，埃里克·华莱士，埃里克·里特，伊万·梅斯，范·王，费利佩·佩特罗斯基·苏奇，菲利普·拉索，弗洛伦西亚·莱奥尼，福ivos·茨米普尔拉斯，弗朗西斯·宋，弗雷德里克·冯·洛曼，弗雷迪·苏利特，杰夫·萨尔蒙，吉安巴蒂斯塔·帕拉桑多洛，吉尔达斯·沙博，赵·Grace，格雷格·布罗克曼，纪尧姆·勒克莱尔，哈迪·萨拉曼，鲍海铭，盛浩，哈特·安德林，赫萨姆·巴格赫里内扎德，任洪宇，亨特·莱特曼，钟炯元，伊恩·基夫利奇安，伊恩·奥康奈尔，伊恩·奥斯班德，伊格纳西奥·克拉韦拉·吉拉贝特，伊尔杰·阿卡亚，伊利亚·科斯特里科夫，伊利亚·苏茨凯弗，伊琳娜·科夫曼，雅各布·帕乔克西，詹姆斯·伦农，魏·Jason，哈恩·Jean，杰里·特沃，冯·Jiacheng，于·Jiahui，翁·Jiayi，唐·Jie，于·Jieqi，胡安昆·Quiñonero

坎德拉，乔·帕尔默，乔尔·帕里什，约翰内斯·海德克，约翰·霍尔曼，约翰·里佐，乔纳森·戈登，乔纳森·乌埃萨托，乔纳森·沃德，乔斯特·休伊辛加，朱莉·王，凯·陈，凯·肖，卡兰·辛哈尔，卡-
林娜·阮，卡尔·科贝，凯蒂·石，凯拉·伍德，肯德拉·林巴赫，克伦·古-伦伯格，Kevin·刘，Kevin·卢，Kevin·斯通，Kevin·余，拉马·阿赫曼，劳伦·杨，Leo·刘，Leon·马克辛，莱顿·何，利亚姆·费-杜斯，莉莉安·王，林登·李，琳达·麦克卡伦，琳赛·赫德，洛伦茨·库恩，卢卡斯·孔德拉丘克，卢卡兹·凯撒，卢克·梅茨，玛德琳·博伊德，玛雅·特雷巴茨，马纳斯·乔格拉克，马克·陈，马克奥·廷托尔，马森·迈耶，马特·琼斯，马特·考费尔，马克斯·施瓦策，梅根·沙，梅赫梅特·亚特巴兹，Melody·Y·关，

徐梦圆, 严梦圆, 格莱塞·米娅, 陈米安娜, 兰佩·迈克尔, 马勒克·迈克尔, 王米歇尔, 弗拉丁·米歇尔, 麦克劳利·迈克, 帕夫洛夫·米哈伊尔, 王米尔斯, 王明轩, 穆拉蒂·米拉, 巴瓦里安·莫, 罗哈尼内贾德·莫斯塔法, 麦卡利斯·内特, 邱杜尔里·尼尔, 邱杜尔里·尼尔, 瑞德·尼克, 泰扎克·尼科拉斯, 布朗·诺姆, 奈赫姆·奥夫, 博伊科·奥列格, 穆尔克·奥列格, 韦金斯·奥利维亚, 赵帕特里克, 阿什伯恩·保罗, 伊兹迈洛夫·帕维尔, 祖科夫·彼得, 戴亚斯·拉切尔, 阿罗拉·拉胡尔, 林·兰德尔, 罗佩斯·贡蒂约·拉法, 高恩·拉兹, 米亚拉·莱阿, 莱克·莱马, 黄锦年, 加格·里瑟姆, 布朗·罗宾, 詹姆斯·罗什安, 舒·瑞, 邱·瑞安, 贾因·萨奇, 阿尔特曼·山姆, 托伊泽·山姆, 托耶尔·山姆, 米塞伦迪诺·塞缪尔, 阿格拉瓦尔·桑迪尼, 埃尔南德斯·圣地亚哥, 贝克·萨莎, 麦金尼·斯科特, 姚斯科蒂, 赵盛嘉, 胡盛立, 山塔卡尔·希巴尼, 雅各布·夏尔曼, 张舒媛, 傅思源, 帕帕伊·斯宾塞, 林·斯蒂夫, 巴拉吉·苏吉尔, 桑吉夫·苏万什, 西多尔·西莫恩, 布罗达·塔尔, 克拉克·艾丹, 王涛, 高登·泰勒, 桑德斯·泰德, 帕特瓦德汉·杰卡尔, 索特奥·蒂博, 德格里·托马斯, 德姆森·托马斯, 郑天昊, 加里波夫·蒂姆尔, 汤姆

斯塔西，特拉皮特·班萨尔，特雷弗·克里奇，特洛伊·彼得森，蒂娜·埃尔翁杜，瓦莱丽·齐，维内特·科萨鲁杜，文尼摩纳哥，维奇·庞，弗拉基米尔·福梅克，郑伟义，周文达，韦斯·麦卡贝，沃伊切赫·扎雷姆巴，扬·杜波依斯，于海瑞，陈一宁，杨昌，白宇，何宇辰，张宇辰，王云云，邵正，以及李卓航。Openai o1系统卡，2024年。URL https://arxiv.org/abs/2412.16720。

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. Kimi k1.5: Scaling reinforcement learning with llms, 2025. URL https://arxiv.org/abs/2501.12599.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL https://github.com/huggingface/open-r1.

曾伟豪，黄宇珍，刘倩，刘伟，何克青，马泽君，以及何俊贤。Simplerl-zoo：

Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL https://arxiv.org/abs/2503.18892.

Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero. https://github.com/Jiayi-Pan/TinyZero, 2025. Accessed: 2025-01-24.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024. URL https://arxiv.org/abs/2409.12122.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021a. URL https://arxiv.org/abs/2110.14168.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL https://arxiv.org/abs/2411.15124.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025. URL https://arxiv.org/abs/2503.20783.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. arXiv:2309.06180.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021b. arXiv:2110.14168.

David Dohan, Winnie Xu, Aitor Lewkowycz, Jacob Austin, David Bieber, Raphael Gontijo Lopes, Yuhuai Wu, Henryk Michalewski, Rif A. Saurous, Jascha Sohl-dickstein, Kevin Murphy, and Charles Sutton. Language model cascades, 2022. URL https://arxiv.org/abs/2207.10342.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (rest) for language modeling, 2023. URL https://arxiv.org/abs/2308.08998.

Joshua Achiam. Spinning up in deep reinforcement learning. 2018a.

Nathan Lambert. Reinforcement learning from human feedback, 2024. URL https://rlhfbook.com.

Sheldon M Ross. *Simulation*. academic press, 2022.

Thomas Degris, Martha White, and Richard S. Sutton. Off-policy actor-critic, 2013. URL https://arxiv.org/abs/1205.4839.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Joshua Achiam. Simplified ppo-clip objective, 2018b. URL https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

NTT123. Grpo-zero. https://github.com/policy-gradient/GRPO-Zero, 2025. 访问时间：2025-05-22。